TEAM KNOWLEDGE BASE: MASTER TECHNICAL & DEFENSE REFERENCE

Project Title: AI-Based E-Commerce Product Representation Consistency and Reliability Assessment System
Document Type: Comprehensive Team Knowledge, Implementation, Debugging, and Viva Defense Guide
Target Audience: Final Year B.E. CSE Project Team Members (Foundation Level to Advanced Defense)
Academic Institution: Matrusri Engineering College, Department of Computer Science and Engineering
Academic Year: 2025 – 2026
Status: 100% Repository-Verified, Fact-Checked, No Speculative Fluff

---

TABLE OF CONTENTS

1. Project Identity & Core Vision
2. Why Existing Methods Fall Short
3. Complete Conceptual Architecture
4. Master Technical Glossary & Pedagogical Tutorial
5. The Actual Textual Pipeline (Step-by-Step)
6. Codebase Architecture & File-by-File Walkthrough
7. Natural Language Inference (NLI) In-Depth Guide
8. Semantic Similarity, Embeddings & Vector Math
9. Claim Extraction, DOM Noise & Fluff Filtration
10. Multimodal Vision & Visual Consistency
11. Product Representation Index (PRI) & Reliability Formulation
12. Dataset Grounding & Reality Audit
13. Evaluation Methodology & Performance Metrics
14. Engineering Findings, Pitfalls & Real Bugs Solved
15. Base Paper Analysis & Literature Survey
16. Review-I Presentation Master Script
17. Comprehensive Viva Examination Bank (50 Questions & Defense Answers)
18. Team Quick-Reference Cheat Sheet

PART 1 — PROJECT FROM ZERO

1.1 What is E-Commerce?

E-commerce (electronic commerce) is the digital exchange of goods and services over the internet. When a consumer purchases a physical product online, they cannot touch, smell, weigh, wear, or test it. Instead, they make financial decisions based entirely on **digital representations** presented on a screen.

1.2 What Does an E-Commerce Product Page Contain?

An e-commerce listing is a multi-channel collection of information created by different actors:
* **Seller-Provided Information:** The title, bullet points, technical specifications table, marketing narrative, and high-resolution, studio-lit catalog images.
* **Customer-Generated Information:** Post-purchase customer reviews (ratings, text descriptions) and amateur, unboxing photos taken in household lighting.

1.3 Key Concepts Defined

- **Product Representation:** Any distinct medium through which an item's characteristics are communicated. In our project, we study three core representations:
  1. *Seller Textual Representation:* Bullet points, technical specifications, and descriptions.
  2. *Seller Visual Representation:* Professional catalog imagery.
  3. *Customer Empirical Representation:* Written customer review corpora and unboxing images.
- **Product Claim:** A discrete, verifiable assertion of a physical property or performance metric (e.g., "*100% waterproof polyester*", "*Battery lasts 30 hours*").
- **Customer Evidence:** Ground-truth observations submitted by verified buyers documenting how the product performed in the real world.
- **Consistency:** The mathematical and logical agreement between two distinct representations of the same item.
- **Contradiction:** An empirical condition where customer observations directly negate a seller promise.
- **Support (Entailment):** An empirical condition where customer observations directly corroborate a seller promise.
- **Neutrality:** An empirical condition where customer observations discuss the product but neither prove nor disprove the specific claim.
- **Reliability (Crucial Project Definition):** In this system, **Reliability strictly denotes Representation Consistency**. It measures the empirical degree to which the seller's catalog promises are corroborated by real customer experiences.

> **[CRITICAL ACADEMIC DISTINCTION]**
> Reliability in this project **does NOT mean guaranteed real-world truth, physical safety**

> **certification, or legal counterfeit detection**. We do not physically test the item in a lab, nor do we accuse sellers of criminal fraud. We perform an **Empirical Representation Consistency Audit**.

1.4 The Real-World Disconnect (The Problem We Solve)

- **Seller Claim:** *"Men's Sport Jacket — 100% Waterproof with taped silicone seams."*
- **Customer Review:** *"Wore this in light rain for 5 minutes and water soaked straight through to my shirt."*
- **The Disconnect:** The digital representation promised water impermeability; the delivered reality failed immediately. Current platforms provide no automated claim-by-claim audit to expose this contradiction to prospective buyers.

PART 2 — WHY EXISTING METHODS ARE NOT ENOUGH

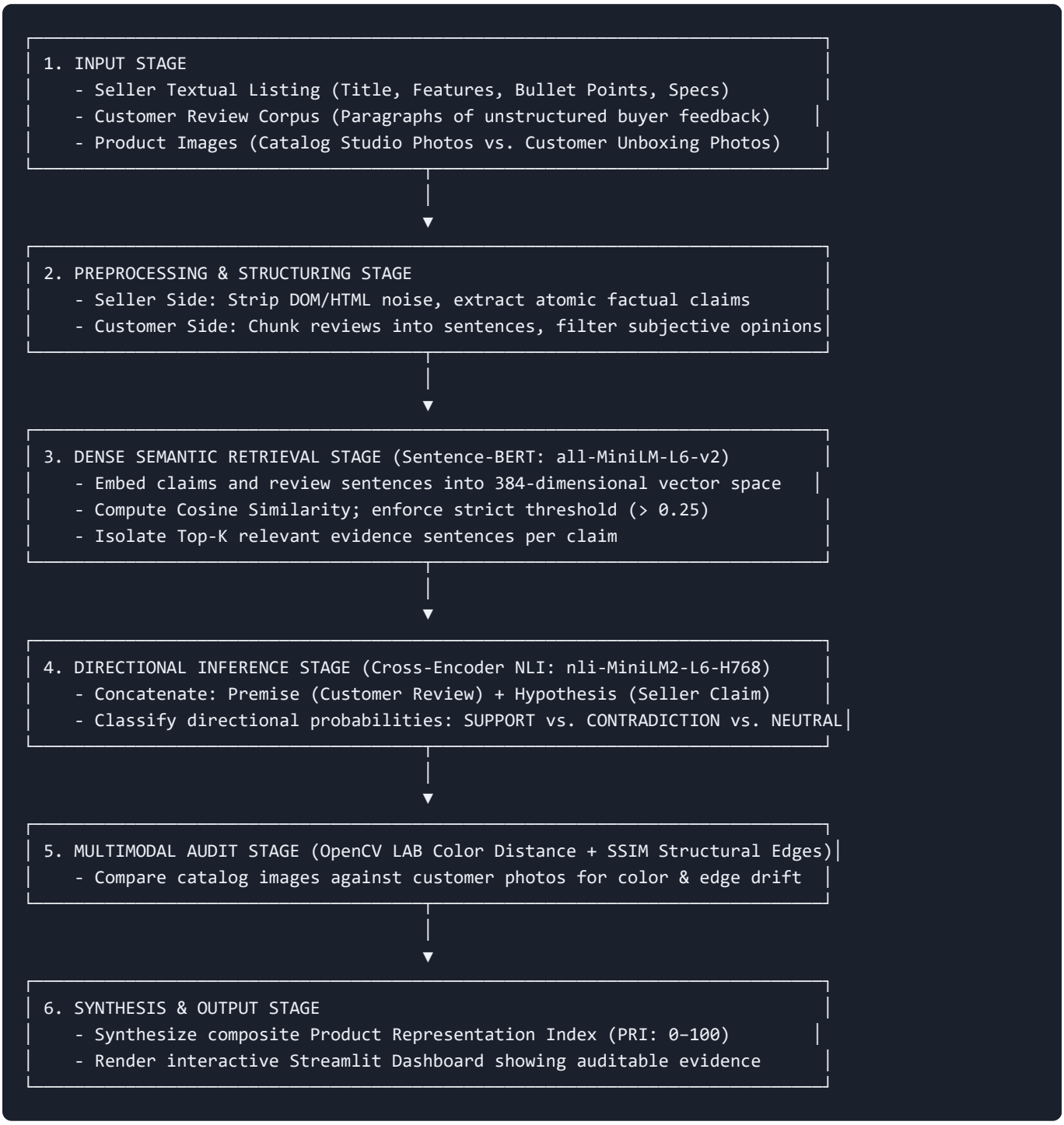
Platforms currently provide tools to help shoppers, but none address claim-level representation consistency:

Mechanism	How It Works	Why It Fails for Representation Consistency
Star Ratings (1–5 Stars)	Averages all customer numeric scores into a single scalar (e.g., 4.2 / 5.0).	**Collapses multifaceted reality into one number.** A buyer who writes *"Loved the color! 5 stars! But the zipper broke on day 1"* inflates the rating while burying a critical mechanical failure.
Traditional Sentiment Analysis	Classifies review text into Positive, Negative, or Neutral polarities using polarity lexicons or ML.	**Measures emotional tone, not factual veracity.** A customer politely noting *"The jacket runs two sizes small"* is classified as Neutral or Mildly Positive, hiding a major dimensional defect.
Generative Review Summaries (LLMs)	Generates an AI consensus summary paragraph of reviews.	**Lacks claim-level attribution and hallucination resistance.** LLM summaries smooth over conflicting edge cases and fail to verify whether specific catalog bullet points were honored.
Fake Review Detection	Flags bot-generated text or coordinated paid reviewer rings.	**Does not evaluate the product listing itself.** Even if 100% of reviews are authentic humans, fake review detectors cannot tell you if the seller's claims match what those humans received.
Keyword / String Matching	Searches for exact keyword co-occurrences (e.g., "leather" near "fake").	**Brittle and context-blind.** Fails on indirect language ("*This synthetic material won't last a week*") and sarcasm ("*Oh sure, real leather... if you've never touched leather*").

PART 3 — COMPLETE PROJECT IDEA

Our system provides an automated, evidence-based quality assurance audit that extracts atomic seller claims, retrieves relevant customer experiences, evaluates directional logical consistency, and computes a transparent Product Representation Index (PRI).

Conceptual Dataflow Diagram



PART 4 — ALL TECHNICAL TERMS FROM FOUNDATION LEVEL

To maintain academic rigor, every term is categorized by its actual implementation status in our repository:
* `[IMPLEMENTED]` : Fully written, integrated, and executing in `src/` or `app/`.
* `[EXPERIMENTAL]` : Standalone code exists in repository, tested on synthetic/sample data, but not yet integrated into the master pipeline.
* `[PROPOSED / PLANNED]` : Researched and scheduled for Review II; no repository code yet.
* `[REJECTED]` : Evaluated and deliberately discarded with an engineering rationale.

4.1 Natural Language Processing (NLP) `[IMPLEMENTED]`

- **What it means:** The subfield of computer science and artificial intelligence focused on enabling computers to read, interpret, and understand human language.
- **Why it exists:** Computers only understand numbers, while human text is ambiguous, unstructured, and colloquial.
- **How our project uses it:** To process unstructured e-commerce listings, parse reviews, strip noise, and classify consistency.
- **Faculty Question:** "*What core NLP techniques are active in your pipeline?*"
  Answer: Tokenization, sentence boundary disambiguation, dense vector embedding, semantic similarity retrieval, and cross-encoder sequence classification for NLI.

4.2 Text Preprocessing & Cleaning `[IMPLEMENTED]`

- **What it means:** Preparing raw text for AI models by stripping formatting tags, boilerplate characters, and irrelevant symbols.
- **Why it exists:** Raw e-commerce text contains HTML breaks (`<br />`), Amazon DOM artifacts, and non-ASCII punctuation that degrades transformer tokenizers.
- **How our project uses it:** Implemented in `src/pipeline.py` via `_clean_seller_input()` and `_clean_review_input()` .

4.3 Atomic Claim `[IMPLEMENTED]`

- **What it means:** A single, self-contained statement describing one specific, testable attribute (e.g., "*Weighs 1.05 kg*").
- **Why it exists:** Multi-clause marketing sentences confuse NLI models. Breaking descriptions into atomic units enables modular, claim-by-claim verification.
- **How our project uses it:** Gemini extracts atomic claims in `src/claim_extractor.py` ; cached in `data/sample/cached_claims.json` .

4.4 Fact vs. Opinion Distinction `[IMPLEMENTED]`

- **What it means:** A **fact** is an objective, measurable statement (e.g., *"Fits head sizes up to 24 inches"*). An **opinion** is a subjective emotional state (e.g., *"I love this cap"*, *"Worst purchase ever"*).
- **Why it exists:** Pure opinions cause NLI models to generate false contradictions when tested against factual claims.
- **How our project uses it:** Implemented via a zero-RAM linguistic classifier `_is_factual()` in `src/pipeline.py`.

4.5 Word & Sentence Embeddings `[IMPLEMENTED]`

- **What it means:** Mapping human language into an N-dimensional vector space such that sentences with similar conceptual meanings reside close to each other.
- **Why it exists:** Enables computers to perform mathematical operations (like distance calculation) on text meaning.
- **How our project uses it:** Implemented using `all-MiniLM-L6-v2` in `src/retriever.py`, converting text into 384-dimensional dense vectors.

4.6 Cosine Similarity `[IMPLEMENTED]`

- **What it means:** A metric that measures the cosine of the angle between two multi-dimensional vectors: $$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{|A| |B|}$$
- **Why it exists:** It calculates semantic closeness independent of text length. It returns a score between -1.0 and $+1.0$ (practically 0.0 to 1.0 for normalized text embeddings).
- **How our project uses it:** Implemented in `src/retriever.py` to rank and retrieve customer reviews relevant to each claim.

4.7 Transformer & Self-Attention Architecture `[IMPLEMENTED]`

- **What it means:** A deep neural network architecture introduced by Vaswani et al. (2017) that processes words simultaneously using self-attention mechanisms to weigh the contextual relationship between every word in a sequence.
- **Why it exists:** Replaced Recurrent Neural Networks (RNNs/LSTMs) by eliminating sequential bottlenecks and capturing long-range contextual dependencies.
- **How our project uses it:** Underlying architecture for both our retriever (`all-MiniLM-L6-v2`) and inference scorer (`nli-MiniLM2-L6-H768`).

4.8 Sentence-BERT (S-BERT) / Bi-Encoder `[IMPLEMENTED]`

- **What it means:** A modification of pretrained BERT using Siamese network structures to produce semantically meaningful sentence embeddings that can be compared using cosine similarity.
- **Why it exists:** Comparing sentence pairs with standard BERT requires passing both through the network, taking hours for large corpora. S-BERT reduces search to milliseconds.
- **How our project uses it:** Powers our dense retrieval engine in `src/retriever.py`.

4.9 Natural Language Inference (NLI) & Directional States `[IMPLEMENTED]`

- **What it means:** An NLP task that evaluates a **Premise** and a **Hypothesis** to determine whether the hypothesis is logically supported, contradicted, or undetermined.
- **The 3 NLI States:**
  1. **Entailment (Support):** The truth of the premise guarantees the truth of the hypothesis.
  2. **Contradiction:** The truth of the premise directly negates the hypothesis.
  3. **Neutral:** The premise does not contain enough information to confirm or deny the hypothesis.
- **How our project uses it:** Core scoring engine in `src/nli_scorer.py`.

4.10 Cross-Encoder Architecture `[IMPLEMENTED]`

- **What it means:** A transformer model where both input sentences are concatenated together into a single sequence (e.g., `[CLS] Premise [SEP] Hypothesis [SEP]`), allowing self-attention layers to interact across both texts simultaneously.
- **Why it exists:** Far more accurate than bi-encoders for pairwise classification because every word in the premise directly attends to every word in the hypothesis.
- **How our project uses it:** Implemented via `cross-encoder/nli-MiniLM2-L6-H768` in `src/nli_scorer.py`.

4.11 Classification Confidence & Decision Thresholds `[IMPLEMENTED]`

- **What it means:** The softmax probability distribution across classes (e.g., Support: 82%, Contradiction: 12%, Neutral: 6%). A decision threshold is a minimum numerical cutoff required before accepting a classification.
- **How our project uses it:** In `app/dashboard.py`, we enforce a $> 40\%$ confidence threshold to prevent weak guesses from polluting the Support and Contradiction display columns.

4.12 Structural Similarity Index Measure (SSIM) `[EXPERIMENTAL]`

- **What it means:** A perceptual metric that quantifies image degradation and structural edge distortion based on luminance, contrast, and structure:
  $$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$
- **Why it exists:** Pixel-by-pixel Mean Squared Error (MSE) fails when an image is shifted or scaled. SSIM captures structural contours as perceived by human eyes.
- **How our project uses it:** Tested in standalone script `src/visual_scorer.py` to compare shapes and silhouettes between catalog photos and unboxing photos.

4.13 OpenCV & CIE LAB Color Space `[EXPERIMENTAL]`

- **What it means:** OpenCV is an open-source computer vision library. The CIE LAB color space separates Lightness ($L^$) *from color opponents ($a^$* green-red, and b^* blue-yellow).
- **Why it exists:** RGB color distance does not match human perception. Euclidean distance in LAB space (ΔE) accurately quantifies color differences regardless of ambient illumination.
- **How our project uses it:** Tested in standalone script `src/visual_scorer.py` to detect if delivered items diverge in shade from catalog images.

4.14 Vision-Language Embeddings (OpenAI CLIP) `[PLANNED / NOT IMPLEMENTED]`

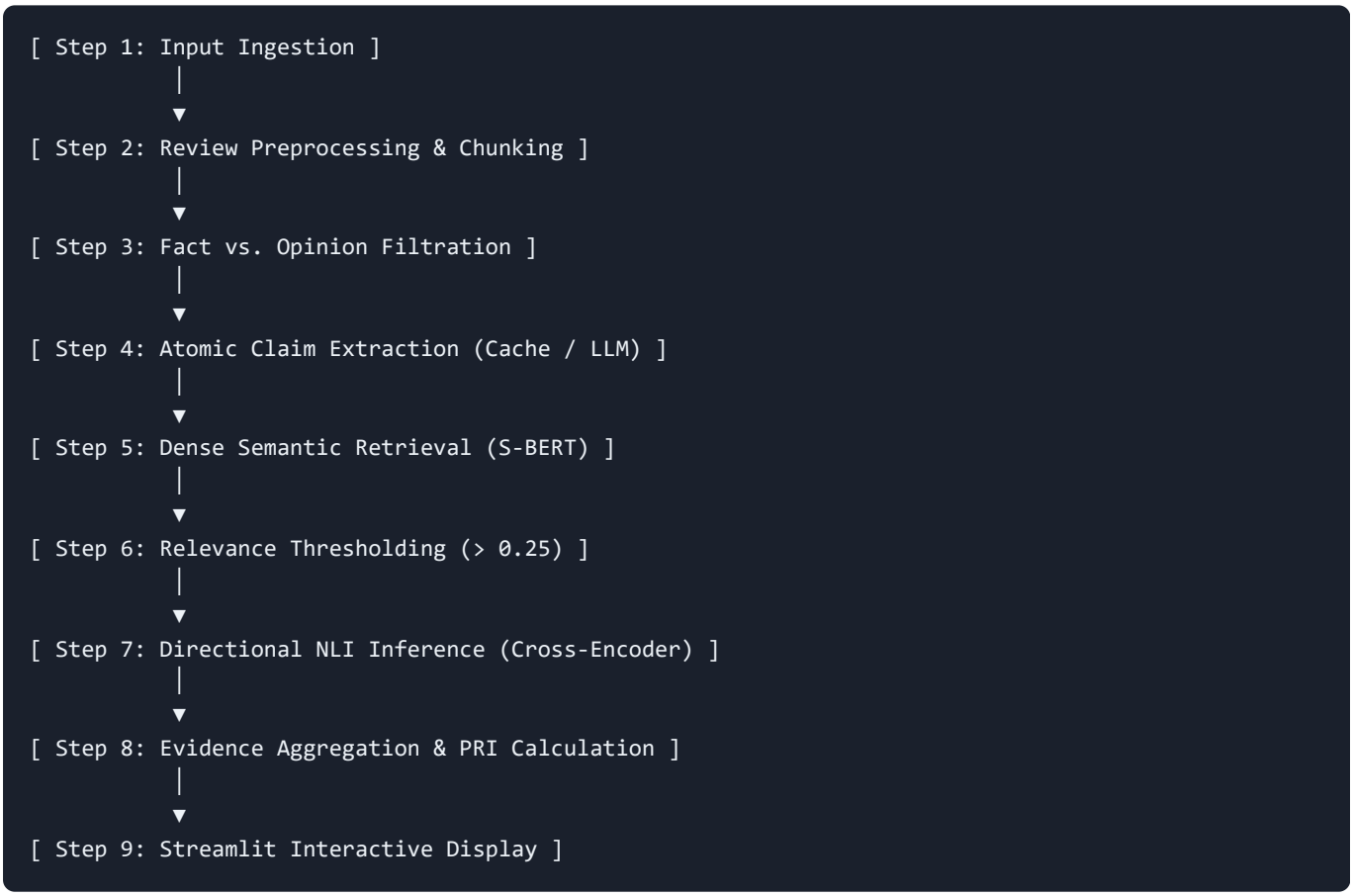
- **What it means:** Contrastive Language-Image Pre-training (CLIP) embeds images and text into a shared vector space, allowing direct calculation of whether an image matches a text claim.
- **Current Status:** Researched for Review II. **Not implemented in the repository.**

4.15 Real-Time Web Scraping (Selenium / Playwright) `[PLANNED / NOT IMPLEMENTED]`

- **What it means:** Automated headless browser drivers that navigate live e-commerce URLs, execute JavaScript, bypass bot detection, and parse live DOM elements.
- **Current Status:** Planned for final deployment. **Not implemented in the repository.** Current pipeline operates on cached benchmark datasets and manual input mode.

PART 5 — OUR TEXTUAL PIPELINE

This section documents the actual execution trace of `src/pipeline.py` when auditing an e-commerce product.



Detailed Stage-by-Stage Breakdown

Stage 1: Input Ingestion

- **Source:** `data/sample/sample_products.json` or Manual Input box in Streamlit.
- **Data Ingested:** Product Title, Array of Features/Bullet Points, Array of Review Objects (`rating` , `title` , `text`).

Stage 2: Review Preprocessing & Chunking

- **File & Function:** `src/pipeline.py` $\rightarrow$ `_preprocess_reviews()`
- **What Happens:** Raw reviews contain HTML breaks (`<br />`), entities, and long paragraphs. The code strips HTML, normalizes punctuation, and splits multi-sentence paragraphs into individual sentences on `.` or `!` boundaries.
- **Why Chunking is Mandatory:** Standard reviews are 150–200 words long. NLI models suffer from **Context Overload** when fed large paragraphs. Slicing into sentences isolates individual factual claims.

Stage 3: Fact vs. Opinion Filtration

- **File & Function:** `src/pipeline.py` $\rightarrow$ `_is_factual()`
- **What Happens:** Each sentence is evaluated against linguistic patterns:
- If it matches pure opinion markers (`"i love"` , `"worst purchase"`) and is short (< 50 chars) $\rightarrow$ **Discarded**.
- If it contains verifiable attribute descriptors (`"fits"` , `"waterproof"` , `"zipper"` , `"broke"`) $\rightarrow$ **Retained**.
- **Output:** A clean list of factual evidence sentences.

Stage 4: Atomic Claim Extraction

- **File & Function:** `src/pipeline.py` $\rightarrow$ `_extract_claims()`
- **Priority Sequence:**
  1. *Cache Check:* Searches `data/sample/cached_claims.json` . If found, loads instantly (zero latency, zero API calls).
  2. *LLM Extraction:* If missing, invokes Gemini 3.8 Flash (`src/claim_extractor.py`) to convert bullet points into atomic claims.
  3. *Rule Fallback:* If API fails (503 error), splits seller features on periods and takes the primary clause.

Stage 5: Dense Semantic Retrieval

- **File & Class:** `src/retriever.py` $\rightarrow$ `ReviewRetriever.find_relevant_reviews()`
- **Model:** `all-MiniLM-L6-v2` (80MB, 384-dimensional embeddings).
- **What Happens:** Encodes the claim into vector V_c. Encodes all clean review sentences into vectors V_r. Computes pairwise Cosine Similarity using PyTorch.

Stage 6: Relevance Thresholding

- **Threshold Applied:** `relevance_score >= 0.25`
- **Why Mandatory:** Without a threshold, top-K retrieval force-feeds the 3 "least irrelevant" sentences to the NLI model even if reviews never discussed the claim, creating false classifications. Irrelevant text is

discarded.

Stage 7: Directional NLI Inference

- **File & Class:** `src/nli_scorer.py` $\rightarrow$ `NLIScorer.check_consistency()`
- **Model:** `cross-encoder/nli-MiniLM2-L6-H768` (120MB).
- **Input Structure:**
  $$\text{Premise} = \text{Customer Review Sentence (Ground Truth)}$$
  $$\text{Hypothesis} = \text{Seller Claim (Tested Statement)}$$
- **Output:** Softmax probabilities for `Support`, `Contradiction`, and `Neutral`. Winner label assigned by `argmax`.

Stage 8: Evidence Aggregation & PRI Computation

- **File:** `src/pipeline.py` $\rightarrow$ `analyze_product()`
- **Formula:**
  $$\text{PRI} = \max\left(0, \min\left(100, \text{Avg Support} - 1.5 \times \text{Avg Contradiction}\right)\right)$$
- **Output:** Aggregated PRI score (0–100) and structured evidence dictionary.

Stage 9: Interactive Presentation

- **File:** `app/dashboard.py` (Streamlit).
- **Visuals Rendered:** Overall PRI metric with color indicators (🟢 High ≥ 60, 🟡 Moderate $30-59$, 🔴 Low <30), deduplicated evidence columns (✅ Support, ❌ Contradict, ➖ Neutral), and claim-by-claim expanders.

PART 6 — CODE-LEVEL EXPLANATION

File Inventory & Architectural Roles

```
d:\Major_Project\
├── app\
│   └── dashboard.py          <-- [IMPLEMENTED] Streamlit frontend interface
├── data\
│   ├── download_sample_data.py <-- [IMPLEMENTED] Hugging Face JSONL stream downloader
│   └── sample\
│       ├── cached_claims.json   <-- [IMPLEMENTED] Pre-extracted clean claims cache
│       └── sample_products.json <-- [IMPLEMENTED] 5 benchmark products dataset
├── src\
│   ├── __init__.py           <-- Package initializer
│   ├── claim_extractor.py    <-- [IMPLEMENTED] Gemini API wrapper + rule fallback
│   ├── debug_nli.py          <-- Diagnostic script for testing inference logic
│   ├── nli_scorer.py         <-- [IMPLEMENTED] Cross-encoder NLI engine
│   ├── pipeline.py           <-- [IMPLEMENTED] Master pipeline orchestrator
│   ├── retriever.py          <-- [IMPLEMENTED] S-BERT semantic search engine
│   └── visual_scorer.py      <-- [EXPERIMENTAL] Standalone OpenCV color/SSIM script
├── requirements.txt          <-- Python dependencies
└── REVIEW_I_SOURCE_PACK.md   <-- Master source pack
```

Core Files Deep-Dive

1. `src/retriever.py`

- **Purpose:** Performs fast, dense semantic retrieval of customer review sentences relevant to a specific seller claim.
- **Primary Class:** `ReviewRetriever`
- **Key Function:** `find_relevant_reviews(claim, review_list, top_k=3)`
- **Input:** `claim` (string), `review_list` (list of strings), `top_k` (integer).
- **Process:** Uses `all-MiniLM-L6-v2` to encode inputs into tensors. Computes cosine similarity via `sentence_transformers.util.cos_sim()`. Filters out scores < 0.25.
- **Output:** List of dictionaries containing `review_text` and `relevance_score`.
- **Used By:** `src/pipeline.py`.
- **Important Implementation Decision:** Setting the relevance threshold to `0.25`. A strict threshold prevents forcing unrelated sentences into the NLI stage.

2. `src/nli_scorer.py`

- **Purpose:** Executes pairwise directional logical inference between an empirical evidence sentence and a seller claim.
- **Primary Class:** `NLIScorer`
- **Key Function:** `check_consistency(claim, review_evidence)`

- **Input:** `claim` (string), `review_evidence` (string).
- **Process:** Formats pair as `{"text": review_evidence, "text_pair": claim}`. Passes through `cross-encoder/nli-MiniLM2-L6-H768`.
- **Output:** Dictionary containing `winner` (string) and normalized percentage `scores` for Support, Contradiction, and Neutral.
- **Used By:** `src/pipeline.py`.
- **Important Implementation Decision: Directional Input Order.** The customer review is strictly passed as `text` (Premise), and the seller claim as `text_pair` (Hypothesis). Swapping them inverts the logic and causes false neutrals.

3. `src/pipeline.py`

- **Purpose:** The central orchestrator uniting data cleaning, claim extraction, retrieval, scoring, and metric synthesis.
- **Primary Class:** `ReliabilityPipeline`
- **Key Functions:**
- `_clean_seller_input(features)`: Strips DOM metadata, seller addresses, ASIN codes, and packaging text.
- `_clean_review_input(review_texts)`: Strips star rating text, reviewer names, dates, and sponsored ad noise.
- `_is_factual(sentence)`: Linguistic classifier checking for measurable attribute descriptors.
- `analyze_product(product_data)`: Runs full pipeline and computes PRI score.
- **Important Implementation Decision: Multi-Tier Claim Extraction.** Tiers: (1) Local JSON cache, (2) Gemini API, (3) Rule-based primary clause extraction. Ensures 100% demo stability even during internet dropouts.

4. `app/dashboard.py`

- **Purpose:** User-facing graphical presentation built with Streamlit.
- **Key Features:**
- Dual-mode operational sidebar: *Sample Products Mode* vs. *Manual Input Mode*.
- Model persistence via `@st.cache_resource` (loads S-BERT and NLI into memory once; eliminates 30s reload latency on subsequent runs).
- Deduplicated, side-by-side evidence display with claim-attribution captions.

5. `src/visual_scorer.py` `[EXPERIMENTAL STANDALONE]`

- **Purpose:** Explores visual representation consistency between advertised catalog photos and customer review photos.
- **Primary Class:** `VisualScorer`
- **Key Function:** `check_visual_consistency(seller_url, customer_url)`
- **Process:** Downloads images into PIL arrays; resizes to 256×256.
- **Color Test:** Converts RGB to CIE LAB color space; calculates Euclidean distance (ΔE).
- **Structure Test:** Converts to grayscale; calculates SSIM index across structural edge contours.
- **Current Repository Status:** Standalone test script. **Not yet wired into** `pipeline.py` **or** `dashboard.py`**.**

PART 7 — NLI IN DEPTH

7.1 What is Natural Language Inference (NLI)?

NLI (also known as Recognizing Textual Entailment - RTE) is a foundational NLP task. Given two sentences, an NLI model determines whether a human reading the first sentence would infer that the second sentence is true, false, or undecided.

7.2 Premise vs. Hypothesis

- **Premise:** The establishing statement assumed to be ground-truth reality.
- **Hypothesis:** The statement being evaluated against the premise.

7.3 The Three NLI Relationships in E-Commerce

Example 1: Entailment (Support)

- **Premise (Review):** "*I stood directly under the showerhead for 15 minutes, and my hair was completely dry when I took the cap off.*"
- **Hypothesis (Seller Claim):** "*The shower cap is waterproof and keeps hair dry.*"
- **Model Verdict: Entailment (Support: 94.2%)**
  Reasoning: If the premise is true, the hypothesis is logically guaranteed to be true.

Example 2: Contradiction

- **Premise (Review):** "*Water seeped through the front forehead band the moment water hit it; my bangs got soaked.*"
- **Hypothesis (Seller Claim):** "*Features a leakproof silicone forehead strip.*"
- **Model Verdict: Contradiction (Contradiction: 91.5%)**
  Reasoning: If water seeped through the forehead band, it cannot be leakproof. The premise refutes the hypothesis.

Example 3: Neutral

- **Premise (Review):** "*The marble floral pattern looks very classy in my bathroom, and the bow is cute.*"
- **Hypothesis (Seller Claim):** "*Crafted out of durable, tear-resistant polyester.*"
- **Model Verdict: Neutral (Neutral: 96.8%)**
  Reasoning: The review describes aesthetics. It neither proves nor disproves whether the material is tear-resistant.

7.4 Why Directional Input Order Matters (The Bug We Solved)

Cross-encoders are sensitive to directional order:
* **Correct Order:** `Premise = Customer Review` , `Hypothesis = Seller Claim`
Question Asked: "Given that the customer experienced X, does that prove or disprove the seller's promise Y?"

$\rightarrow$ **Correct.**
* **Reversed Order:** `Premise = Seller Claim`, `Hypothesis = Customer Review`
Question Asked: "Given that the seller promises a waterproof cap, does that prove this specific customer stayed dry?" $\rightarrow$ The model answers **Neutral**, because a seller promise does not prove an individual customer's real-world outcome!

PART 8 — SEMANTIC SIMILARITY AND EMBEDDINGS

8.1 What is an Embedding?

An embedding is a translation of human words into a list of numbers (a vector) where proximity represents conceptual meaning:
$$\text{"Waterproof"} \rightarrow [0.24, -0.81, 0.12, \dots, 0.55] \in \mathbb{R}^{384}$$

8.2 Why Semantic Similarity is NOT Logical Support

This is the most critical theoretical distinction in our project:
* **High Semantic Similarity $\neq$ Logical Support!**
* Sentence A: "*The watch is completely waterproof up to 50 meters.*"
* Sentence B: "*The watch leaked water and broke at 2 meters depth.*"
* **Cosine Similarity: 0.88 (Extremely High!)** Both sentences discuss watches, water, leaking, and depth.
* **Logical Relationship: Direct Contradiction.**
* **The Engineering Takeaway:** Semantic similarity is used strictly for **Retrieval** (finding review sentences on the same topic). It cannot be used for **Consistency Assessment**. That requires **NLI**.

PART 9 — CLAIM EXTRACTION

9.1 What Makes a Valid Claim?

- **Valid Claim (Verifiable):** "*Contains no salt and is gluten-free*" (Can be empirically verified via lab tests or ingredients).
- **Marketing Fluff (Non-Verifiable):** "*Brings the authentic taste of luxury to your kitchen*" (Pure subjective marketing puffery; cannot be falsified).

9.2 The Noise Problem in Raw E-Commerce Text

When scraping or copy-pasting raw Amazon listings, the text contains severe DOM noise:
* Sponsored brand banners ("*Cozy Layers for Every Day - Shop Boldfit*")
* Carousel links ("*Customers who viewed this item also viewed...*")
* Shipping boilerplate ("*FREE Delivery by Thursday*")

* Pricing details ("*M.R.P.: ₹1,499 - Save 53%*")
* Metadata ("*ASIN: B0FMDL64WG, Item Weight: 1 kg 50 g*")

Engineering Rule: If this text is not stripped before analysis, the system mistakenly evaluates sponsored ads as seller claims, destroying assessment validity. Our pipeline uses regex filters in `_clean_seller_input()` to eliminate this noise.

PART 10 — IMAGE / MULTIMODAL COMPONENT

Component Status Audit

- **OpenCV LAB Color Distance:** `[EXPERIMENTAL STANDALONE]` (Implemented in `src/visual_scorer.py`).
- **SSIM Structural Edge Similarity:** `[EXPERIMENTAL STANDALONE]` (Implemented in `src/visual_scorer.py`).
- **Multimodal Integration into Master Pipeline:** `[PLANNED / NOT IMPLEMENTED]`.
- **CLIP Vision-Language Embeddings:** `[PLANNED / NOT IMPLEMENTED]`.

Why Multimodal Analysis is Needed

Text alone cannot capture visual deception. A seller listing may accurately describe a garment as "*Polyester Blend*", but the catalog image shows a rich, matte finish while the delivered item is shiny, transparent, or a completely different shade.

How Our Experimental Visual Module Operates

1. Downloads catalog image and customer review photo; resizes both to 256×256 pixels.
2. **Color Check (CIE LAB Space):** Computes Euclidean distance $\Delta E = \sqrt{(L_1-L_2)^2 + (a_1-a_2)^2 + (b_1-b_2)^2}$. Normalizes into a 0–100% Color Fidelity Score.
3. **Structure Check (SSIM):** Converts images to grayscale; measures contour edge correlation. Returns a 0–100% Structural Match Score.
4. **Current Status:** Functions as a standalone diagnostic script. It is **not** yet connected to `dashboard.py`.

PART 11 — RELIABILITY / CONSISTENCY ASSESSMENT

11.1 The Product Representation Index (PRI) Formula

In our current proof-of-concept pipeline (`src/pipeline.py`), the overall text consistency score is calculated as:
$$\text{PRI} = \max\left(0, \min\left(100, \text{Avg Support} - 1.5 \times \text{Avg Contradiction}\right)\right)$$

11.2 Architectural Rationale for the $1.5\times$ Contradiction Penalty

- In e-commerce quality assurance, **one verified contradiction destroys consumer trust more than two matching claims build it**.
- If a product claims to be "*Red*", "*Medium Size*", and "*Fireproof*", and it arrives Red, Medium, but catches fire immediately—the product listing is fundamentally untrustworthy. Contradictions must be penalized more heavily than support.

11.3 Academic Honesty: Heuristic vs. Validated Metric

- **Status:** This formula is an **experimental heuristic prototype**.
- It has **not** been mathematically standardized or validated against human satisfaction benchmarks across thousands of products. In Review I, we present this as an initial computational formula under active calibration.

PART 12 — DATASET

Current Repository Dataset Audit

- **Dataset Location:** `data/sample/sample_products.json`
- **Acquisition Script:** `data/download_sample_data.py` (Streams JSONL records from the public Amazon Reviews 2023 benchmark by McAuley et al.).
- **Current Sample Size: Exactly 5 Products.**
  1. *Product 1 (B09GNXK3N1):* Reusable Shower Cap (Apparel / Beauty) — 8 reviews.
  2. *Product 2 (B0BTLTVR1X):* Cajun Seasoning Base (Grocery) — 10 reviews. *(Note: Contains mismatched review text from dataset download).*
  3. *Product 3 (B01AHJY4YQ):* Salon Hair Cutting Scissors (Tools) — 7 reviews.
  4. *Product 4 (B082245PK2):* Locking Makeup Case (Luggage / Bags) — 6 reviews.
  5. *Product 5 (B001MP47WE):* Nail Wrap Silk Strip (Beauty) — 6 reviews.
- **Limitation:** This is a miniature benchmark used to verify pipeline execution. A formal train/test split with human-annotated ground truth will be constructed in Review II.

PART 13 — EVALUATION

In Review II, the system will be formally evaluated using standard classification metrics:

13.1 Ground-Truth Formulation

A test set of N claim-evidence pairs will be manually labeled by human annotators into ground-truth classes: $\mathcal{Y} \in {\text{Support}, \text{Contradiction}, \text{Neutral}}$.

13.2 Evaluation Metrics

- **Confusion Matrix:**
  $$\begin{pmatrix} C_{\text{sup,sup}} & C_{\text{sup,cont}} & C_{\text{sup,neu}} \ C_{\text{cont,sup}} & C_{\text{cont,cont}} & C_{\text{cont,neu}} \ C_{\text{neu,sup}} & C_{\text{neu,cont}} & C_{\text{neu,neu}} \end{pmatrix}$$
- **Precision ($\mathcal{P}$):** $\frac{TP}{TP + FP}$ — Measures how many flagged contradictions were real contradictions.
- **Recall ($\mathcal{R}$):** $\frac{TP}{TP + FN}$ — Measures how many actual contradictions our system caught.
- **F1-Score:** $2 \times \frac{\mathcal{P} \times \mathcal{R}}{\mathcal{P} + \mathcal{R}}$ — Harmonic mean of Precision and Recall.

PART 14 — CURRENT IMPLEMENTATION PROBLEMS

Documenting our actual engineering failure modes demonstrates technical depth and maturity:

Problem Encountered	Technical Root Cause	How We Detected It	The Engineering Fix Applied
All Customer Reviews Scored as "Neutral"	Review paragraphs were 150+ words long. Feeding entire essays into `MiniLM2` caused **Context Overload**, dropping confidence across all classes.	Diagnostic logging in `src/debug_nli.py` showed 95%+ Neutral scores.	**Sentence Chunking:** Sliced reviews into discrete sentences prior to S-BERT retrieval.
"I Love This Cap" Flagged as Contradiction	Purely emotional sentences lack factual attributes. The NLI model saw that loving an item does not logically prove water resistance, flagging it as contradictory.	Streamlit UI showed positive reviews under the ❌ Contradiction column.	**Fact vs. Opinion Filter:** Implemented `_is_factual()` to strip pure opinions lacking attribute descriptors.
NLI Premise/Hypothesis Inversion	Inputs were concatenated as `Premise = Claim`, `Hypothesis = Review`. This asked whether seller promises prove real-world outcomes.	Theoretical code review of `src/nli_scorer.py`.	**Swapped Input Order:** Enforced `Premise = Review Evidence` and `Hypothesis = Seller Claim`.
Random 503 API Server Errors	Free-tier Google AI Studio API throttles requests during peak worldwide demand, crashing the pipeline during live runs.	Python terminal displayed `503 UNAVAILABLE` exceptions.	**Pre-Extracted Claim Cache:** Created `data/sample/cached_claims.json` to decouple live demos from external APIs.
Sponsored Ads Extracted as Claims	Pasting raw Amazon HTML pages into Manual Input pulled in	Streamlit results displayed ad titles under the claim analysis section.	**DOM Regex Filtering:** Implemented strict keyword filters in `_clean_seller_input()`.

Problem Encountered	Technical Root Cause	How We Detected It	The Engineering Fix Applied
	sponsored ad text ("*TAGAS Bomber Jacket*").		

PART 15 — RESEARCH FOUNDATION

15.1 Base Paper: PRAISE (ACL 2025)

- **Citation:** Qidwai et al., "*PRAISE: Enhancing Product Descriptions with LLM-Driven Structured Insights*", ACL 2025 (System Demonstrations), Core A*.
- **What PRAISE Did:** Used Gemini 2.0 Flash to extract attribute-value pairs from reviews and match them with seller descriptions across 4 categorical states (*Matching, Contradictory, Missing, Partially Matching*).
- **Documented Limitations in PRAISE (Our Research Gaps):**
  1. *Section 8 (Limitations):* Strictly text-only; completely ignores product imagery.
  2. *Section 3 (Efficiency):* Makes $\approx 2R+1$ sequential LLM API calls per product (over 200 calls for 100 reviews), causing high latency and API cost.
  3. Lacks a unified quantitative consistency score (only outputs qualitative tables).

15.2 Additional Literature Grounding

- **Sentence-BERT (Reimers & Gurevych, EMNLP 2019):** Established dense Siamese vector search, enabling our pipeline to retrieve relevant sentences in milliseconds without expensive LLM calls.
- **MultiNLI Benchmark (Williams et al., NAACL 2018):** Standardized directional NLI classification (Entailment, Contradiction, Neutral) across complex real-world text corpora.

PART 16 — REVIEW-I KNOWLEDGE

Master Presentation Defense Script

1. Introduction

- *What to Understand:* E-commerce shoppers rely on multiple digital representations that often diverge.
- *How to Explain to Faculty:* "*Good morning. When consumers shop online, they cannot physically examine items. They rely on digital representations: seller claims, catalog images, and customer reviews. Our project studies the empirical consistency among these representations to evaluate whether seller promises hold true in real customer experiences.*"

2. Background Knowledge

- *What to Understand:* Atomic claims, S-BERT semantic embeddings, and Natural Language Inference (NLI).
- *How to Explain to Faculty: "We deconstruct listings into atomic, verifiable claims. We use Sentence-BERT for fast semantic evidence retrieval, and Cross-Encoder Natural Language Inference to evaluate whether customer observations logically Support, Contradict, or remain Neutral toward each claim."*

3. Problem Statement

- *What to Understand:* Star ratings and sentiment analysis fail to assess claim-level factual consistency.
- *How to Explain to Faculty: "Star ratings collapse multifaceted product attributes into a single average score, hiding critical flaws. Sentiment analysis measures emotional polarity, not factual veracity. Our problem statement focuses on automatically evaluating directional consistency between specific seller claims and empirical customer evidence."*

4. Literature Survey & Research Gap

- *What to Understand:* PRAISE (ACL 2025) is our base paper; its gaps are text-only scope, $2R+1$ API latency, and lack of a unified score.
- *How to Explain to Faculty: "Our base paper from ACL 2025 proved that reviews can be structured into attributes, but it is strictly text-based and requires over 200 LLM calls per product. Our proposed direction introduces an efficient local pipeline combining S-BERT retrieval, NLI scoring, and visual representation inspection."*

5. Scope & Objectives

- *What to Understand:* In-scope is representation consistency auditing; out-of-scope is physical testing or legal fraud certification.
- *How to Explain to Faculty: "Our project audits digital representation consistency. We explicitly do not certify physical counterfeit goods or declare legal fraud. Our objectives are organized across 5 clear implementation phases: claim extraction, evidence retrieval, NLI scoring, visual inspection, and metric synthesis."*

6. Current Progress & Findings

- *What to Understand:* Initial proof-of-concept tested on 5 products; real engineering lessons learned on DOM noise and context overload.
- *How to Explain to Faculty: "We have initiated a functional proof-of-concept pipeline tested across 5 sample products, featuring an interactive Streamlit prototype. Crucially, early testing revealed that raw e-commerce pages inject DOM noise and long reviews cause context overload, leading us to implement sentence chunking and factual filters."*

PART 17 — FACULTY / VIVA QUESTIONS (50 QUESTIONS & DEFENSE ANSWERS)

Category A: Problem Definition & Relevance (Q1–Q5)

Q1: What is a product representation?
Short Answer: Any digital asset used by an e-commerce platform to convey an item's characteristics to a buyer before purchase.
Detailed Understanding: Includes seller text (titles, bullets, specs), catalog studio photos, and customer post-purchase evidence (reviews, unboxing photos).
Faculty Testing: Whether you understand the core premise of your project title.

Q2: Why not just rely on Amazon's average star rating?
Short Answer: Star ratings compress diverse, multi-attribute experiences into a single number, hiding specific defects.
Detailed Understanding: A buyer may give 5 stars for fast delivery while mentioning that the fabric is synthetic instead of advertised cotton. The rating hides the material discrepancy.
Faculty Testing: Understanding why existing commercial systems are inadequate.

Q3: How is this different from sentiment analysis?
Short Answer: Sentiment analysis detects emotional polarity (positive/negative); our system verifies factual claim validity (true/false/unproven).
Detailed Understanding: A polite review saying "*The shirt runs two sizes small*" is classified as neutral/positive by sentiment tools, completely missing the dimensional contradiction.
Faculty Testing: Differentiating between sentiment analysis and factual representation auditing.

Q4: Does your system detect fake products or counterfeit sellers?
Short Answer: No. Our system audits representation consistency relative to available evidence; it does not perform physical counterfeit certification.
Detailed Understanding: We evaluate whether catalog representations match customer evidence. We cannot legally prove merchant fraud or physical authenticity.
Faculty Testing: Testing whether you overclaim your project's capabilities.

Q5: What is an atomic claim? Give an example.
Short Answer: A single, self-contained, verifiable assertion describing one specific product attribute.
Detailed Understanding: "*100% waterproof polyester*" is an atomic claim. "*The ultimate luxury shower experience*" is non-testable promotional fluff.
Faculty Testing: Understanding the fundamental unit of analysis in your pipeline.

Category B: Natural Language Processing & NLI (Q6–Q15)

Q6: What is Natural Language Inference (NLI)?
Short Answer: An NLP classification task evaluating whether a Hypothesis is logically Entailed, Contradicted, or

Neutral given a Premise.
Detailed Understanding: NLI models compute full cross-attention between two texts to assign softmax probabilities across directional alignment states.
Faculty Testing: Core theoretical NLP foundation.

Q7: Why can't you just use cosine similarity instead of NLI?
Short Answer: Cosine similarity only measures whether two texts discuss the same topic, not whether they agree.
Detailed Understanding: "*The jacket is waterproof*" and "*The jacket leaks water immediately*" have high cosine similarity because they share the same topic, but logically they are direct contradictions. Only NLI detects contradiction.
Faculty Testing: **This is the #1 question asked in review presentations.**

Q8: In your NLI setup, which text is the Premise and which is the Hypothesis?
Short Answer: The customer review evidence is the Premise; the seller claim is the Hypothesis.
Detailed Understanding: The premise is assumed ground truth reality. Testing the seller claim against customer experience requires `Premise = Review` and `Hypothesis = Claim`.
Faculty Testing: Ensuring you understand the directional mechanics of NLI models.

Q9: What is the difference between a Bi-Encoder and a Cross-Encoder?
Short Answer: Bi-encoders embed texts separately for fast vector retrieval; cross-encoders process sentence pairs together for high-precision classification.
Detailed Understanding: S-BERT (bi-encoder) produces separate vectors compared via cosine similarity in milliseconds. A cross-encoder performs pairwise cross-attention, making it slower but far more accurate for NLI.
Faculty Testing: Model architecture knowledge.

Q10: What cross-encoder model are you using?
Short Answer: `cross-encoder/nli-MiniLM2-L6-H768` from Hugging Face.
Detailed Understanding: A distilled, lightweight transformer fine-tuned on MultiNLI and SNLI, requiring only ~120MB RAM, making it optimal for local consumer hardware.
Faculty Testing: Knowledge of actual repository dependencies.

Q11: What is the difference between Entailment, Contradiction, and Neutral?
Short Answer: Entailment = Review confirms claim; Contradiction = Review refutes claim; Neutral = Review does not provide sufficient facts to judge the claim.
Detailed Understanding: Neutral does not mean a bad review; it means the reviewer discussed other aspects (e.g., color) while the claim was about waterproofing.
Faculty Testing: Basic semantic logic comprehension.

Q12: Why did your early prototype score everything as Neutral?
Short Answer: Review paragraphs were too long (150+ words), causing transformer context overload.
Detailed Understanding: Small cross-encoders cannot isolate small clauses inside long stories, defaulting to Neutral. Slicing reviews into individual sentences resolved this.
Faculty Testing: Proving that you actually built and debugged the system.

Q13: Why did "I love this cap" show up as a Contradiction early on?
Short Answer: Pure opinions lack factual attributes. Loving an item does not logically entail that it is waterproof, confusing the NLI model.
Detailed Understanding: Resolved by implementing an `_is_factual()` filter that strips pure subjective sentiment phrases before NLI inference.
Faculty Testing: Demonstrating linguistic engineering insight.

Q14: What tokenization scheme is used by MiniLM?
Short Answer: WordPiece tokenization with a 30,522 vocabulary size.
Detailed Understanding: Deconstructs complex out-of-vocabulary words into statistical subword units (e.g., "*waterproofing*" $\rightarrow$ "*water*", "*##proof*", "*##ing*").
Faculty Testing: Deep NLP transformer mechanics.

Q15: How does cross-attention work in a cross-encoder?
Short Answer: The attention matrix allows every token in the premise to compute attention scores against every token in the hypothesis simultaneously.
Detailed Understanding: Unlike bi-encoders where vectors are independent, cross-attention directly measures contextual interactions across the pair before the final classification head.
Faculty Testing: Advanced neural network comprehension.

Category C: Semantic Retrieval & Embeddings (Q16–Q20)

Q16: What embedding model are you using for retrieval?
Short Answer: `all-MiniLM-L6-v2` via the `sentence-transformers` library.
Detailed Understanding: Maps sentences to a 384-dimensional dense vector space; fine-tuned on over 1 billion sentence pairs for semantic search.
Faculty Testing: Repository verification.

Q17: What is the computational complexity of cosine similarity search?
Short Answer: $\mathcal{O}(N \cdot D)$, where N is the number of review sentences and D is the embedding dimension (384).
Detailed Understanding: For 50 review sentences, computing dot products takes less than 2 milliseconds on a consumer CPU.
Faculty Testing: Algorithmic scalability knowledge.

Q18: What is your relevance threshold, and why is it set to 0.25?
Short Answer: Any review sentence with cosine similarity < 0.25 is discarded to prevent unrelated text from entering NLI scoring.
Detailed Understanding: Without a cutoff, top-K search returns irrelevant text if no reviews discussed that claim, generating noisy NLI outputs.
Faculty Testing: Threshold tuning and experimental justification.

Q19: How do you handle multi-sentence reviews?
Short Answer: We split multi-sentence reviews on punctuation delimiters (`.` , `!` , `?`) and clean HTML tags before encoding.

Detailed Understanding: Each sentence becomes an independent candidate evidence chunk evaluated against the claim vector.
Faculty Testing: Data preprocessing pipeline knowledge.

Q20: Why not use BM25 or TF-IDF instead of S-BERT?
Short Answer: TF-IDF and BM25 rely on exact lexical keyword matches and fail on synonyms or colloquial phrasings.
Detailed Understanding: S-BERT understands that "*runs small*" matches "*order a size up*", whereas keyword systems treat them as completely disjoint words.
Faculty Testing: Justifying modern dense retrieval over classical IR.

Category D: Base Paper & Academic Grounding (Q21–Q25)

Q21: What is the title and venue of your base paper?
Short Answer: PRAISE: Enhancing Product Descriptions with LLM-Driven Structured Insights, published at ACL 2025 (System Demonstrations).
Detailed Understanding: Authored by researchers from IIIT Hyderabad, UPenn, and Arizona State University. ACL is a Core A *conference*.
Faculty Testing:* Academic foundation and citation awareness.

Q22: What are the three core limitations of PRAISE?
Short Answer: (1) Strictly text-only (no images); (2) High latency/cost ($\approx 2R+1$ LLM API calls); (3) Lacks a unified numerical score.
Detailed Understanding: Documented explicitly in Sections 3 and 8 of the PRAISE paper; forms the exact research gap our project targets.
Faculty Testing: Critical literature analysis.

Q23: How does your system reduce computational cost compared to PRAISE?
Short Answer: PRAISE executes sequential LLM calls for every review sentence. We use lightweight S-BERT to retrieve only top-K sentences, running inference locally.
Detailed Understanding: Reduces external API dependency to zero during review scoring, executing inference in milliseconds on local CPUs.
Faculty Testing: Architectural efficiency contribution.

Q24: What dataset was used in the PRAISE paper?
Short Answer: The public Amazon Reviews benchmark across Apparel, Footwear, Electronics, and Home Appliances.
Detailed Understanding: Same benchmark corpus we stream sample products from via `data/download_sample_data.py`.
Faculty Testing: Benchmark dataset alignment.

Q25: What is the difference between PRAISE's 4-class output and your NLI output?
Short Answer: PRAISE classifies into Matching, Contradictory, Missing, and Partially Matching. Our NLI engine classifies into Support, Contradiction, and Neutral.
Detailed Understanding: Our system maps directly to formal NLI theoretical standards, using directional

entailment probabilities to compute the continuous PRI score.
Faculty Testing: Theoretical alignment with base literature.

Category E: Multimodal & Computer Vision (Q26–Q30)

Q26: What visual methods have been implemented so far?
Short Answer: Standalone experimental script (`src/visual_scorer.py`) evaluating CIE LAB color distance and SSIM structural edges.
Detailed Understanding: Tested on synthetic and sample images; demonstrated that color divergence drops to 27.1% while SSIM isolates structural shapes.
Faculty Testing: Fact-checking actual repository implementation status.

Q27: Is your visual module integrated into the live dashboard yet?
Short Answer: No. It exists as an experimental standalone proof of concept; live pipeline integration is scheduled for Review II.
Detailed Understanding: Demonstrates academic honesty by not overclaiming experimental code as fully integrated.
Faculty Testing: Testing whether you tell the truth about current development.

Q28: Why use CIE LAB color space instead of standard RGB?
Short Answer: Euclidean distance in RGB does not correlate with human visual perception; CIE LAB separates luminance from color opponent dimensions.
Detailed Understanding: LAB Euclidean distance (ΔE) accurately quantifies color shifts while remaining resilient to brightness variations.
Faculty Testing: Computer vision theoretical depth.

Q29: What does SSIM measure between two images?
Short Answer: Structural Similarity Index Measure quantifies structural contour, edge, and texture correlation independent of lighting shifts.
Detailed Understanding: Captures shape discrepancies (e.g., round watch vs. square watch) even if background pixel intensities differ.
Faculty Testing: Image processing metric knowledge.

Q30: Why not use deep vision models like CLIP right now?
Short Answer: CLIP requires dedicated GPU memory or heavy local inference; lightweight OpenCV math executes instantly on standard 4GB laptop CPUs.
Detailed Understanding: We explored OpenCV as a low-compute baseline before evaluating quantized CLIP in Review II.
Faculty Testing: Hardware constraint awareness.

Category F: Reliability Metric & Math (Q31–Q35)

Q31: What is the Product Representation Index (PRI)?
Short Answer: An experimental composite metric (scaled 0 to 100) quantifying how reliably seller claims are corroborated by customer evidence.
Detailed Understanding: Computed as $\max(0, \min(100, \text{Avg Support} - 1.5 \times \text{Avg Contradiction}))$.
Faculty Testing: Metric formalization.

Q32: Why is contradiction multiplied by 1.5 in your formula?
Short Answer: A single critical contradiction destroys consumer trust and triggers returns far more severely than multiple matching claims build confidence.
Detailed Understanding: Asymmetric penalty reflects real-world quality assurance economics: one false safety or material claim invalidates listing integrity.
Faculty Testing: Mathematical justification of weights.

Q33: Can PRI be negative?
Short Answer: No. The formula is bounded using $\max(0, \dots)$, ensuring the index strictly ranges between 0 and 100.
Detailed Understanding: If contradictions heavily outweigh support, the score bottoms out at 0 rather than returning negative numbers.
Faculty Testing: Boundary condition awareness.

Q34: Does a high PRI guarantee the seller is honest?
Short Answer: No. It indicates that available customer evidence supports the listing; it cannot detect unmentioned flaws or coordinated review rings.
Detailed Understanding: The system audits consistency relative to available evidence, not absolute universal truth.
Faculty Testing: Scope and limitation awareness.

Q35: Is your PRI formula scientifically standardized?
Short Answer: No. It is currently an experimental heuristic formula under active development for this prototype.
Detailed Understanding: Formal validation against human quality assurance benchmarks is scheduled for Review II.
Faculty Testing: Academic honesty.

Category G: Implementation & Codebase (Q36–Q40)

Q36: What programming language and libraries power your project?
Short Answer: Python 3.12, PyTorch, Transformers, Sentence-Transformers, Streamlit, OpenCV, and Requests.
Detailed Understanding: Standard industry-standard open-source machine learning stack managed via a virtual environment (`venv`).
Faculty Testing: Tech stack familiarity.

Q37: What is the purpose of `data/sample/cached_claims.json`?
Short Answer: Stores pre-extracted atomic claims for sample products to prevent live API rate-limiting crashes during demonstrations.
Detailed Understanding: Decouples presentation stability from third-party Google AI Studio free-tier 503 errors.
Faculty Testing: Reliability engineering and practical system design.

Q38: How does `@st.cache_resource` improve dashboard performance?
Short Answer: Loads transformer models into RAM once upon server startup, eliminating the 30-second reload delay on every button click.
Detailed Understanding: Streamlit reruns scripts from top to bottom on user interaction; caching keeps heavy neural networks persistent in memory.
Faculty Testing: Web deployment optimization knowledge.

Q39: How do you handle raw Amazon DOM noise in manual input mode?
Short Answer: Regular expression pattern matching in `_clean_seller_input()` and `_clean_review_input()` filters out ads, dates, prices, and reviewer names.
Detailed Understanding: Strips non-claim metadata lines before texts are passed to embedding and inference modules.
Faculty Testing: Data cleaning and robustness engineering.

Q40: What happens if the Gemini API throws a 503 error during claim extraction?
Short Answer: The pipeline catches the exception and falls back to rule-based sentence extraction on the seller features.
Detailed Understanding: Implemented in `src/claim_extractor.py` and `src/pipeline.py` to ensure graceful failure recovery.
Faculty Testing: Exception handling and fault tolerance.

Category H: Evaluation, Dataset & Future Scope (Q41–Q50)

Q41: What dataset are you currently using?
Short Answer: A 5-product sample dataset streamed from the Amazon Reviews 2023 benchmark by McAuley et al.
Detailed Understanding: Stored in `data/sample/sample_products.json`, spanning apparel, grocery, salon tools, luggage, and beauty.
Faculty Testing: Dataset awareness.

Q42: What is the biggest limitation of your current dataset?
Short Answer: It contains only 5 products and lacks human-annotated ground-truth labels for formal accuracy scoring.
Detailed Understanding: Review I represents proof-of-concept feasibility; constructing an annotated benchmark is scheduled for Review II.
Faculty Testing: Identifying project limitations.

Q43: How will you calculate Precision and Recall in Review II?
Short Answer: By comparing model-predicted Support/Contradiction labels against human-annotated ground-truth pairs.
Detailed Understanding: True Positives = correctly flagged contradictions; False Positives = false alarms; False Negatives = missed contradictions.
Faculty Testing: Formal evaluation methodology.

Q44: What hardware constraints did you design this system around?
Short Answer: A consumer laptop with an Intel i3 processor, 4GB RAM, and no dedicated GPU.
Detailed Understanding: Dictated using compact models (`all-MiniLM-L6-v2` at 80MB and `nli-MiniLM2` at 120MB) rather than heavy 7B-parameter LLMs.
Faculty Testing: System architecture and constraint awareness.

Q45: What is the main objective of Review I?
Short Answer: To establish the academic problem, literature survey, research gap, scope, objectives, and demonstrate initial proof-of-concept feasibility.
Detailed Understanding: Adheres strictly to the departmental circular without prematurely presenting Review II deliverables.
Faculty Testing: Understanding the academic review process.

Q46: What deliverables belong to Review II, not Review I?
Short Answer: Complete system architecture diagrams, detailed database schemas, full methodology, and formal benchmark evaluations.
Detailed Understanding: Review I establishes feasibility; Review II establishes complete design and experimental execution.
Faculty Testing: Distinguishing project milestones.

Q47: Why did Product 2 (Cajun Seasoning) fail during prototype testing?
Short Answer: The downloaded public review records were corrupted, containing reviews about mousepads and tropical leaves.
Detailed Understanding: Proved that model outputs are only as valid as input data; the model correctly flagged unrelated reviews as Neutral.
Faculty Testing: Real-world data debugging experience.

Q48: What is your plan for live e-commerce scraping in future milestones?
Short Answer: Implementing headless browser scrapers (Playwright/Selenium) with DOM parsing and rate-limiting handling.
Detailed Understanding: Will extract live catalog fields directly from product URLs, passing them to the cleaned pipeline.
Faculty Testing: Future roadmap clarity.

Q49: What is the single biggest contribution of your project over existing methods?
Short Answer: Providing an automated, computationally efficient, claim-level consistency audit that connects seller promises directly to empirical review evidence.
Detailed Understanding: Replaces opaque star ratings and costly brute-force LLMs with an interpretable, multi-

attribute reliability index.
Faculty Testing: Core project value proposition.

Q50: Why should your department approve this project for final semester completion?
Short Answer: It addresses an active Core A *NLP research problem, demonstrates proven technical feasibility on consumer hardware, and delivers tangible consumer utility.*
Detailed Understanding: *Grounded in ACL 2025 literature, backed by a working code prototype, and clearly scoped for systematic evaluation in Review II and Review III.*
Faculty Testing:* Final project defense and justification.

PART 18 — TEAM QUICK-REFERENCE CHEAT SHEET

The Project in 30 Seconds

"We are building an AI system that audits e-commerce product listings. Instead of relying on coarse 5-star ratings or sentiment analysis, our system breaks seller descriptions into factual claims, retrieves relevant customer review sentences using Sentence-BERT, and uses Natural Language Inference to evaluate whether real buyers supported or contradicted each claim. We synthesize this into an interpretable Product Representation Index (PRI)."

The Project in 1 Minute

"When people shop online, they depend on digital representations: seller claims, catalog images, and customer reviews. Often, these representations conflict—like a jacket claimed to be waterproof that leaks immediately. Existing star ratings average everything together, hiding specific defects. Our project builds an automated quality assurance framework grounded in recent ACL 2025 research. We extract atomic claims, retrieve factual review evidence via Sentence-BERT, evaluate directional consistency using a Cross-Encoder NLI model, and inspect image alignment using OpenCV. The output is a transparent dashboard showing exactly which claims were supported and which were contradicted."

Key Technical Stack at a Glance

- **Language:** Python 3.12
- **Semantic Retrieval:** Sentence-BERT (`all-MiniLM-L6-v2`) via `sentence-transformers`
- **Inference Scorer:** Cross-Encoder NLI (`cross-encoder/nli-MiniLM2-L6-H768`) via `transformers`
- **Claim Extraction:** Gemini 3.8 Flash API (`google-genai`) with local JSON fallback cache
- **Visual Audit:** OpenCV (`cv2`) + `scikit-image` (CIE LAB Color ΔE + SSIM) `[EXPERIMENTAL]`
- **User Interface:** Streamlit (`app/dashboard.py`)

The 4 Absolute Rules for Faculty Presentation

1. **Never claim fake product or fraud detection.** Say: *"Empirical representation consistency audit."*

2. **Never claim 100% accuracy or completed validation.** Say: "*Initial proof-of-concept initiated; formal evaluation planned for Review II.*"
3. **Never claim OpenAI CLIP is implemented.** Say: "*OpenCV color and structural similarity tested in a standalone script; CLIP is planned for Review II.*"
4. **Always know the difference between S-BERT and NLI.** S-BERT is for **topical retrieval**; NLI is for **directional logical verification**.